

reflexión breve

¿Qué cambió en tu forma de "dar por terminado" el código cuando el veredicto lo decidió un **gate determinista** en vez de tu propio criterio?

Antes daba por terminado el código cuando veía que funcionaba y las pruebas principales pasaban. Con el gate entendí que eso no siempre es suficiente. En mi caso, todo parecía estar bien, pero el build falló porque una clase tenía 57 % de cobertura y se exigía mínimo 80 %.

Esto me hizo darme cuenta de que a veces uno puede pasar por alto ciertos problemas por apuro o porque piensa que no son tan importantes. Ahora considero que el código está terminado cuando existe evidencia de que cumple las pruebas, la cobertura, la seguridad y los requisitos definidos, no solamente cuando yo pienso que ya funciona.

¿Qué pilar te costó más dejar en verde —**pruebas, seguridad o criterios**—, y por qué?

El pilar que más me costó fue seguridad. Tuve varios problemas para configurar Semgrep y conectarlo mediante MCP. Incluso tuve que cambiar la configuración porque inicialmente estaba utilizando una versión que ya estaba obsoleta.

Además, se encontró un problema en la validación del webhook de WhatsApp. Si el secreto no estaba configurado, la aplicación podía aceptar una solicitud sin comprobar su firma. Al principio no era un error evidente porque las pruebas pasaban y la funcionalidad trabajaba normalmente.

Esta parte me costó más porque no se trataba únicamente de corregir código. También tuve que entender cómo funcionaban el token, el servidor MCP, la firma HMAC y la configuración de seguridad de Spring.

¿Para qué te serviría un gate de **Definition of Done** (y el escaneo automático de seguridad vía MCP) en tu equipo real?

En un equipo real, el gate serviría para que todos tengamos una misma definición de lo que significa que una tarea esté terminada. Así no dependeríamos solamente de la opinión de cada desarrollador.

También ayudaría a detectar problemas antes de unir el código a la rama principal. Por ejemplo, podría evitar que se apruebe un cambio con poca cobertura, pruebas fallidas, requisitos sin comprobar o vulnerabilidades importantes.

El escaneo de seguridad mediante MCP sería útil porque permitiría revisar el código automáticamente durante el desarrollo. Esto no reemplazaría completamente la revisión humana, pero funcionaría como una ayuda adicional para encontrar problemas que

Juan Pablo Anton Yagual

podrían pasar desapercibidos. En general, considero que permitiría entregar código con más confianza y reducir errores antes de llegar a producción.